# K. R. MANGALAM UNIVERSITY

## B. TECH CSE (Data Science)

SOET

(School of Engineering &Technology)

ENCS256-ANALYSIS AND DESIGN OF ALGORITHMS


Submitted by-

Name - Sakshi Jain

Roll No. – 2401420034

Submitted to-

Aarti Ma'am

# TASK 1: BACKTRACKING ALGORITHMS

## Code:

```
def is_safe(board,row,col,n):
    for i in range(row):
        if board[i]==col:
            return False

    # Check left diagonal
    i=row-1
    j=col-1
    while i>=0 and j>=0:
        if board[i]==j:
            return False
        i-=1
        j-=1

    # Check right diagonal
    i=row-1
    j=col+1
    while i>=0 and j<n:
        if board[i]==j:
            return False
        i-=1
        j+=1
    return True
def solve_nqueens(board,row,n):
    if row==n:
        print("Solution:",board)
        return
    for col in range(n):
        if is_safe(board,row,col,n):
            board[row]=col
            solve_nqueens(board,row+1,n)    # Recursive call
            board[row]=-1                   # Backtrack
n=6
board = [-1]*n
solve_nqueens(board, 0, n)
```

## Output:

```
Solution: [1, 3, 5, 0, 2, 4]
Solution: [2, 5, 1, 4, 0, 3]
Solution: [3, 0, 4, 1, 5, 2]
Solution: [4, 2, 0, 5, 3, 1]
```

## Observations:

- Backtracking explores all possible solutions by recursively trying different choices and backtracking when a constraint is violated.
- It is useful for constraint satisfaction problems.
- The algorithm may have high time complexity because it checks many possible combinations.

# TASK 2: BRANCH AND BOUND ALGORITHMS

## Code:

```
class Item:
    def __init__(self,value,weight):
        self.value=value
        self.weight=weight
        self.ratio=value/weight
# Bounding function to estimate the maximum possible profit
def bound(level,weight,profit,items,capacity):
    if weight>=capacity:
        return 0
    profit_bound=profit
    j=level+1
    total_weight=weight

    # Add items while capacity allows
    while j<len(items) and total_weight+items[j].weight<=capacity:
        total_weight+=items[j].weight
        profit_bound+=items[j].value
        j+=1

    # Add fraction of next item
    if j<len(items):
        profit_bound+=(capacity-total_weight)*items[j].ratio
    return profit_bound
def knapsack_branch_bound(items,capacity):

    # Sort items by value/weight ratio
    items.sort(key=lambda x:x.ratio,reverse=True)
    max_profit=0

    # Stack stores (level, profit, weight)
    stack=[]
    stack.append((-1,0,0))
    while stack:
        level,profit,weight=stack.pop()
        if level==len(items)-1:
            continue
        level+=1
```

```python
        # Include item
        new_weight=weight+items[level].weight
        new_profit=profit+items[level].value
        if new_weight<=capacity and new_profit>max_profit:
            max_profit=new_profit
        if bound(level,new_weight,new_profit,items,capacity)>max_profit:
            stack.append((level,new_profit,new_weight))

        # Exclude item
        if bound(level,weight,profit,items,capacity)>max_profit:
            stack.append((level,profit,weight))
    return max_profit

# Example
items=[
    Item(60,10),
    Item(100,20),
    Item(120,30)
]
capacity=50
print("Maximum Profit:",knapsack_branch_bound(items,capacity))
```

## Output:

```
Maximum Profit: 220
```

## Observations:

- Branch and Bound reduces the search space by eliminating non-promising branches using a bounding function.
- It is commonly used for optimization problems such as the 0/1 Knapsack problem.
- Compared to brute force, it improves efficiency by pruning unnecessary computations.

# TASK 3: STRING MATCHING ALGORITHMS

## Code:

```
import time

#Naive String Matching
def naive_search(text,pattern):
    n=len(text)
    m=len(pattern)
    comparisons=0
    for i in range(n-m+1):
        for j in range(m):
            comparisons+=1
            if text[i+j]!=pattern[j]:
                break
        else:
            print("Pattern found at index", i)
    return comparisons

#KMP Algorithm
def compute_lps(pattern):
    lps=[0]*len(pattern)
    length=0
    i=1
    while i<len(pattern):
        if pattern[i]==pattern[length]:
            length+=1
            lps[i]=length
            i+=1
        else:
            if length!=0:
                length=lps[length-1]
            else:
                lps[i]=0
                i+=1
    return lps
```

```python
def kmp_search(text,pattern):
    lps=compute_lps(pattern)
    i=0
    j=0
    comparisons=0
    while i<len(text):
        comparisons+=1
        if pattern[j]==text[i]:
            i+=1
            j+=1
        if j==len(pattern):
            print("Pattern found at index",i-j)
            j=lps[j-1]
        elif i<len(text) and pattern[j]!=text[i]:
            if j!=0:
                j=lps[j-1]
            else:
                i+=1
    return comparisons

#Performance Comparison
pattern="ABC"
text_sizes=[50,100,500,1000]
for size in text_sizes:
    text="A"*size+"ABC"
    start=time.time()
    naive_comp=naive_search(text,pattern)
    naive_time=time.time()-start

    start=time.time()
    kmp_comp=kmp_search(text,pattern)
    kmp_time=time.time()-start

    print("\nText Size:",size)
    print("Naive Comparisons:",naive_comp)
    print("Naive Time:",naive_time)
    print("KMP Comparisons:",kmp_comp)
    print("KMP Time:",kmp_time)
```

## Output:

```
Pattern found at index 50
Pattern found at index 50

Text Size: 50
Naive Comparisons: 103
Naive Time: 0.0
KMP Comparisons: 53
KMP Time: 0.0
Pattern found at index 100
Pattern found at index 100

Text Size: 100
Naive Comparisons: 203
Naive Time: 0.0
KMP Comparisons: 103
KMP Time: 0.0
Pattern found at index 500
Pattern found at index 500

Text Size: 500
Naive Comparisons: 1003
Naive Time: 0.0010612010955810547
KMP Comparisons: 503
KMP Time: 0.0
Pattern found at index 1000
Pattern found at index 1000

Text Size: 1000
Naive Comparisons: 2003
Naive Time: 0.0
KMP Comparisons: 1003
KMP Time: 0.0
```

## Observations:

- The Native string matching algorithm checks the pattern at every position of the text, leading to many categories.
- The KMP algorithm reduces redundant comparisons by using the LPS (Longest Prefix Suffix) table.
- As the text size increases, KMP performs better than the Native algorithm.

# TASK 4: EXPERIMENTAL COMPARISON OF STRING MATCHING

## Code:

```
import time
import matplotlib.pyplot as plt

#Naive String Matching
def naive_search(text, pattern):
    n=len(text)
    m=len(pattern)
    comparisons=0
    for i in range(n-m+1):
        for j in range(m):
            comparisons+=1
            if text[i+j]!=pattern[j]:
                break
    return comparisons

#KMP Algorithm
def compute_lps(pattern):
    lps = [0] * len(pattern)
    length=0
    i=1
    while i<len(pattern):
        if pattern[i]==pattern[length]:
            length+=1
            lps[i]=length
            i+=1
        else:
            if length!=0:
                length=lps[length-1]
            else:
                lps[i]=0
                i+=1
    return lps

def kmp_search(text,pattern):
    lps=compute_lps(pattern)
    i=0
    j=0
    comparisons=0
    while i<len(text):
        comparisons+=1
        if pattern[j]==text[i]:
            i+=1
            j+=1
```

```
        if j==len(pattern):
            j=lps[j-1]
        elif i<len(text) and pattern[j]!=text[i]:
            if j!=0:
                j=lps[j-1]
            else:
                i+=1
    return comparisons

#Experimental Comparison
pattern="ABC"
sizes=[100,500,1000,2000,5000]
naive_results=[]
kmp_results=[]
for size in sizes:
    text="A"*size+"ABC"
    naive_comp=naive_search(text, pattern)
    kmp_comp=kmp_search(text, pattern)
    naive_results.append(naive_comp)
    kmp_results.append(kmp_comp)
    print("Text Size:", size)
    print("Naive Comparisons:", naive_comp)
    print("KMP Comparisons:", kmp_comp)
    print()

#Plot Comparison
plt.plot(sizes,naive_results,marker='o',label="Naive Algorithm")
plt.plot(sizes,kmp_results,marker='o',label="KMP Algorithm")
plt.xlabel("Text Size")
plt.ylabel("Number of Comparisons")
plt.title("Comparison of String Matching Algorithms")
plt.legend()
plt.show()
```

## Output:

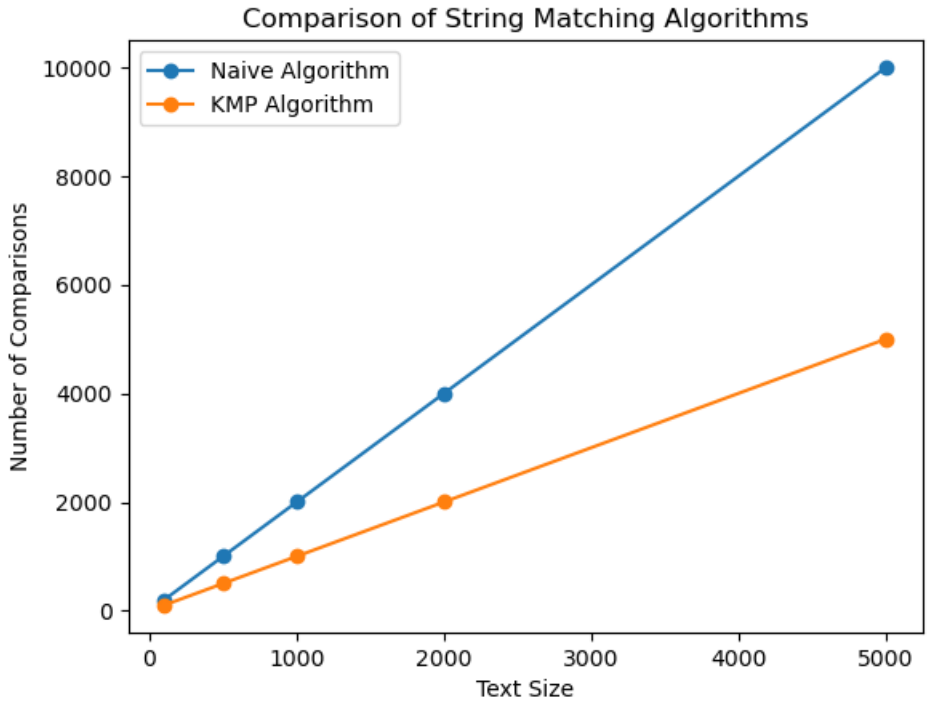
```
Text Size: 100
Naive Comparisons: 203
KMP Comparisons: 103

Text Size: 500
Naive Comparisons: 1003
KMP Comparisons: 503

Text Size: 1000
Naive Comparisons: 2003
KMP Comparisons: 1003

Text Size: 2000
Naive Comparisons: 4003
KMP Comparisons: 2003

Text Size: 5000
Naive Comparisons: 10003
KMP Comparisons: 5003
```




## Observations:

- The number of character comparisons increases with larger text sizes for both algorithms.
- The Native algorithm shows faster growth in comparisons, making it less efficient for large texts.
- The KMP algorithm performs fewer comparisons and run faster, which makes it more suitable for large-scale string searching.

# TASK 5: CONCEPTUAL PROGRAMMING TASK

## Code:

```python
# List of flights with start and end times
flights = [('F1',9,11),('F2',10,12),('F3',12,14),('F4',13,15)]
crew_members = ['C1','C2','C3']        # Available crew members
assignment={}                          # Dictionary to store final assignments
def is_valid(crew,flight,assignment):
    """
    This function checks whether assigning a crew member
    to a flight violates any constraints.
    """
    start,end=flight[1],flight[2]
    for f,c in assignment.items():
        if c==crew:
            s,e=f[1],f[2]
            if not (end<=s or start>=e):
                return False
    return True

def backtrack(i):
    """
    Recursive backtracking function that tries to assign
    flights to crew members one by one.
    """
    if i==len(flights):
        return True
    flight=flights[i]
    for crew in crew_members:
        if is_valid(crew,flight,assignment):
            assignment[flight]=crew
            if backtrack(i+1):
                return True
            del assignment[flight]
    return False

backtrack(0)
print("Final Crew Scheduling:")
for f,c in assignment.items():
    print(f[0],"assigned to",c)
```

## Output:

```
Final Crew Scheduling:
F1 assigned to C1
F2 assigned to C2
F3 assigned to C1
F4 assigned to C2
```

## Observations:

- The backtracking algorithm tries different crew-flight assignments and checks constraints before confirming a valid schedule.
- The is_valid() function ensures that no crew member is assigned overlapping flights, maintaining schedule feasibility.
- If a conflict occurs, the algorithm backtracks and tries another possible assignment, ensuring a correct solution is found.